

ใบงานที่ 3

เรื่อง การทดลองใช้ MSI ออกแบบวงจรเปรียบเทียบ

จัดทำโดย

นาย ศิริพงษ์ บุญประสิทธิ์

รหัส ENGCE200_SEC1

เสนอ

อาจารย์สมนึก สุระธง

ใบงานนี้เป็นส่วนหนึ่งของวิชา

ENGCE200

การออกแบบดิจิทัล

(Digital Systems Design)

หลักสูตร วศ.บ.วิศวกรรมคอมพิวเตอร์

สาขาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา ภาควิชา วิทยาเขต เชียงใหม่

ภาคเรียนที่ 1 ปีการศึกษา 2569

ใบงานที่ 3

ชื่อบทเรียน การทดลองใช้ MSI ออกแบบวงจรเปรียบเทียบ

จุดประสงค์การสอน ปฏิบัติการทดลองใช้ MSI ออกแบบวงจรเปรียบเทียบ

ออกแบบและทดสอบวงจรเปรียบเทียบ โดยใช้ MSI

1. ทฤษฎี

วิธีออกแบบวงจรด้วย Variable-Entering Map

วิธีการออกแบบวงจร โดยใช้ 4 : 1 Multiplex

2. จงออกแบบวงจรเปรียบเทียบ 4 บิต กำหนดให้

2.1 มี Input เป็น A มี 4 บิต และ B มี 4 บิต สามารถเปรียบเทียบ น้อยกว่า,มากกว่า, เท่ากับ

2.2 ใช้ 4 : 1 Multiplex

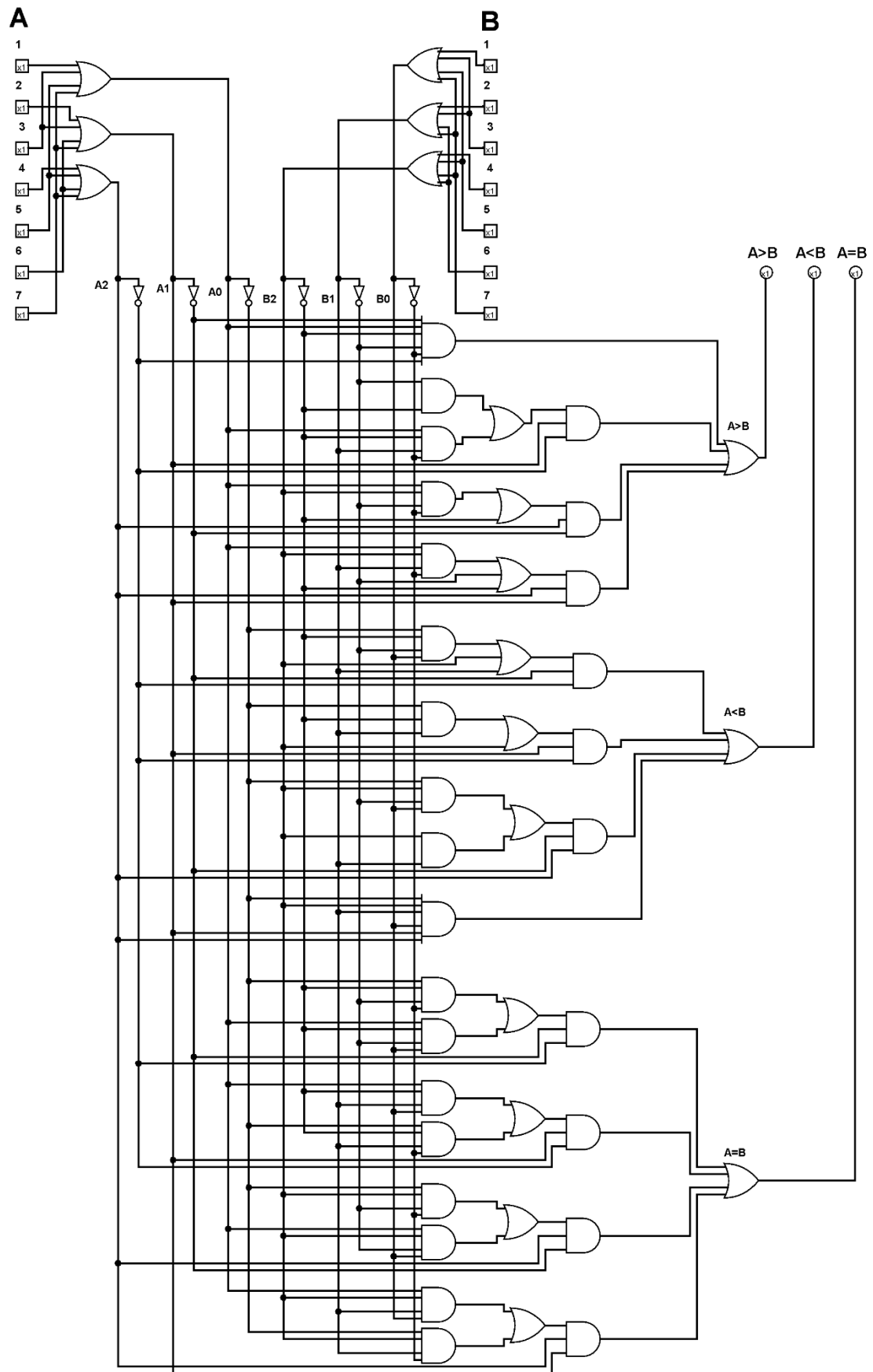
3. ผลการทดลอง

4. สรุปผลการทดลอง

5. คำถามหลังการทดลอง

1. บอกข้อดีและข้อเสียการใช้ MSI ออกแบบวงจร
2. บอกหลักการใช้ Multiplex ออกแบบวงจร
3. เมื่อไรถึงจะใช้หลักการ Variable-Entering Map

A	B	A2	A1	A0	B2	B1	B0	A>B	A<B	A=B
0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	1	0
0	2	0	0	0	0	1	0	0	1	0
0	3	0	0	0	0	1	1	0	1	0
0	4	0	0	0	1	0	0	0	1	0
0	5	0	0	0	1	0	1	0	1	0
0	6	0	0	0	1	1	0	0	1	0
0	7	0	0	0	1	1	1	0	1	0
1	0	0	0	1	0	0	0	1	0	0
1	1	0	0	1	0	0	1	0	0	1
1	2	0	0	1	0	1	0	0	1	0
1	3	0	0	1	0	1	1	0	1	0
1	4	0	0	1	1	0	0	0	1	0
1	5	0	0	1	1	0	1	0	1	0
1	6	0	0	1	1	1	0	0	1	0
1	7	0	0	1	1	1	1	0	1	0
2	0	0	1	0	0	0	0	1	0	0
2	1	0	1	0	0	0	1	1	0	0
2	2	0	1	0	0	1	0	0	0	1
2	3	0	1	0	0	1	1	0	1	0
2	4	0	1	0	1	0	0	0	1	0
2	5	0	1	0	1	0	1	0	1	0
2	6	0	1	0	1	1	0	0	1	0
2	7	0	1	0	1	1	1	0	1	0
3	0	0	1	1	0	0	0	1	0	0
3	1	0	1	1	0	0	1	1	0	0
3	2	0	1	1	0	1	0	1	0	0
3	3	0	1	1	0	1	1	0	0	1
3	4	0	1	1	1	0	0	0	1	0
3	5	0	1	1	1	0	1	0	1	0
3	6	0	1	1	1	1	0	0	1	0
3	7	0	1	1	1	1	1	0	1	0
4	0	1	0	0	0	0	0	1	0	0



Test.vhd

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity test is

Port (

A2, A1, A0 : in STD_LOGIC;

B2, B1, B0 : in STD_LOGIC;

A_more_B : out STD_LOGIC;

A_less_B : out STD_LOGIC;

A_equal_B : out STD_LOGIC

);

end test;

architecture Behavioral of test is

signal L0, L1, L2, L3 : STD_LOGIC;

signal L4, L5, L6, L7 : STD_LOGIC;

signal L8, L9, L10, L11 : STD_LOGIC;

begin

-- A > B

L0 <= (not A2 and not A1) and

(A0 and not B2 and not B1 and not B0);

L1 <= (not A2 and A1) and

((not B2 and not B1) or

(A0 and not B2 and B1 and not B0));

L2 <= (A2 and not A1) and

(B2 or

(A0 and B2 and not B1 and not B0));

L3 <= (A2 and A1) and

(not B2 or not B1 or

(A0 and B2 and B1 and not B0));

-- A < B

L4 <= (not A2 and not A1) and
(B1 or B2 or
(not A0 and not B2 and not B1 and B0));

L5 <= (not A2 and A1) and
(B2 or
(not A0 and not B2 and B1));

L6 <= (A2 and not A1) and
((B2 and B1) or
(not A0 and B2 and not B1 and B0));

L7 <= (A2 and A1) and
(not A0 and B2 and B1 and B0);

-- A = B

L8 <= (not A2 and not A1) and
((not A0 and not B2 and not B1 and not B0) or
(A0 and not B2 and not B1 and B0));

L9 <= (not A2 and A1) and
((A0 and not B2 and B1 and B0) or
(not A0 and not B2 and B1 and not B0));

L10 <= (A2 and not A1) and
((not A0 and B2 and not B1 and not B0) or
(A0 and B2 and not B1 and B0));

L11 <= (A2 and A1) and
((A0 and B2 and B1 and B0) or
(not A0 and B2 and B1 and not B0));

A_more_B <= L0 or L1 or L2 or L3;

A_less_B <= L4 or L5 or L6 or L7;

A_equal_B <= L8 or L9 or L10 or L11;

end Behavioral;

tb_test.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity tb_test is
```

```
end tb_test;
```

```
architecture Behavioral of tb_test is
```

```
    component test
```

```
    Port (
```

```
        A2, A1, A0 : in STD_LOGIC;
```

```
        B2, B1, B0 : in STD_LOGIC;
```

```
        A_more_B  : out STD_LOGIC;
```

```
        A_less_B  : out STD_LOGIC;
```

```
        A_equal_B : out STD_LOGIC
```

```
    );
```

```
end component;
```

```
    signal A2, A1, A0 : STD_LOGIC := '0';
```

```
    signal B2, B1, B0 : STD_LOGIC := '0';
```

```
    signal A_more_B, A_less_B, A_equal_B : STD_LOGIC;
```

```
begin
```

```
    UUT: test
```

```
    port map (
```

```
        A2 => A2,
```

```
        A1 => A1,
```

```
        A0 => A0,
```

```
        B2 => B2,
```

```
        B1 => B1,
```

```
        B0 => B0,
```

```
        A_more_B => A_more_B,
```

```
        A_less_B => A_less_B,
```

```
        A_equal_B => A_equal_B
```

);

stimulus : process

begin

-- A=0, B=0 => A=B

A2<='0'; A1<='0'; A0<='0';

B2<='0'; B1<='0'; B0<='0';

wait for 10 ns;

-- A=5, B=3 => A>B

A2<='1'; A1<='0'; A0<='1';

B2<='0'; B1<='1'; B0<='1';

wait for 10 ns;

-- A=2, B=6 => A<B

A2<='0'; A1<='1'; A0<='0';

B2<='1'; B1<='1'; B0<='0';

wait for 10 ns;

-- A=7, B=7 => A=B

A2<='1'; A1<='1'; A0<='1';

B2<='1'; B1<='1'; B0<='1';

wait for 10 ns;

-- A=4, B=1 => A>B

A2<='1'; A1<='0'; A0<='0';

B2<='0'; B1<='0'; B0<='1';

wait for 10 ns;

-- A=1, B=4 => A<B

A2<='0'; A1<='0'; A0<='1';

B2<='1'; B1<='0'; B0<='0';

wait for 10 ns;

wait;

end process;

end Behavioral;

